

INDEPENDENT VERIFICATION • READ-ONLY

Sage 300 → SMEAssist: what the evidence actually shows

A statement-by-statement re-examination of the prior migration audit, re-derived from the live Sage `IDEDAT` database, the live SMEAssist MySQL schema, and the SMEAssist Java source. Nothing was modified: every statement issued was a `SELECT`, `SHOW` or `DESCRIBE`.

Date 8 Sep 2026 **Sage** SQL Server 2025 • IDEDAT • 1,110 tables **Target** MySQL 8.0.46 • org wonderblues

Window 1 Jan – 30 Apr 2026 **Posture** read-only

00 The four questions

First, a correction about the brief

The file this review was asked to verify —

`Sage300_to_SMEAssist_Historical_Context_for_Verification.md` — **does not exist** anywhere on this machine, and no trace of it appears in git history. Rather than invent its contents, this review verifies the artifact that genuinely matches the description: the untracked `migration-analysis/` package (25 documents, 12,426 lines, written 5–7 Sep 2026 by a prior session), whose `FINAL-REPORT.md` is a Phase 0 audit that ends by naming the eight of its own claims most likely to be wrong. Those became this review's attack list.

What is the real problem we are trying to solve?

It is a completeness and scope problem wearing the costume of a correctness problem. The transformation logic — Indian GST, reverse charge, rate snapping, receipt splits — is demonstrably sound: of 9,376 posted bills, 5,769 match Sage to the paise and only **three** non-reverse-charge documents differ by $\geq ₹1$. That is not what a broken mapping looks like.

What is actually unsolved is that **the migration has moved one narrow slice of the business and nothing else**. Purchase invoices are 34.2% across by count and 13.5% by value — but that single average hides the real shape: AP-direct expense bills are **72.4%** done while the goods/PO stream is at **0.88%**. And every other object is at exactly zero: no sales invoices, no purchase orders, no goods receipts, no deliveries, no payments, no receipts, no credit or debit notes, no opening balances, no

chart of accounts, no journal vouchers. Sage's window holds roughly ₹386 crore of sales that no part of this project touches.

Layered on top are two live hazards: one armed data defect that will understate ₹38 crore the next time the pipeline runs, and one script that deletes every bill in the organisation while its docstring promises it cannot.

What did we previously think the problem was?

Two different prior framings, and they should not be conflated.

The brief's framing — implicit in the 20-phase instruction — treats the flow as unknown and undiscovered: that we must establish whether sales runs `OL → OE` or `MM → OE`, whether a shared item master is the right model, and follow Sage PDF documentation to learn the purchase process. All three premises turn out to be either already settled or factually wrong (see [§ Rejected](#)).

The prior audit's framing was much closer: "the transformation logic is good; its completeness, its scope governance and its safety rails are not." That framing survives this review essentially intact.

What does the actual evidence show?

The prior audit is unusually trustworthy and it reproduces exactly. Its population figures came back to the paise on independent re-derivation — AP-direct 12,781 / ₹1,763,950,260.97 and PO 18,046 / ₹1,942,654,924.63, plus the 1,079 no-distribution documents at ₹295,107,559.76. Its foundational assumption, which it explicitly asked to have checked first, is **confirmed**. Its most reassuring claim survived a hostile re-test.

But its single highest-priority remedy is wrong. The audit correctly identified that the goods path posts source-currency amounts as rupees, and correctly told you to switch to the `ap_amount_hc` column. That column is itself corrupted on at least 174 in-window documents, because the staging extract strips trailing punctuation from invoice numbers and then joins on the damaged key — pulling amounts from a different real invoice, in one proven case from a different financial year. Applying the recommended fix as written would import wrong values.

And two of the brief's central questions dissolve on contact with the data. The `MM` module has 68 tables and **two rows**. The shared item-master question is architecturally correct but empirically almost irrelevant here: of 17,129 purchased and 2,918 sold items, exactly **26** appear in both, because Indian Designs manufactures rather than trades.

What exactly should we implement?

In this order, and not in any other:

1. **Disarm** `work/cleanup_pilot.py` **today**. One careless invocation destroys 9,377 bills. Cost: minutes.
2. **Do not run** `goods-post` **until the currency fix lands** — and fix it by carrying real currency and conversion rate into the fields SMEAssist already has, not by switching to `ap_amount_hc` .
3. **Re-extract the staging mirror on the unstripped invoice number**, then prove the repair by reconciling the mirror against live `APOBL` per document.
4. **Move the crosswalk off one laptop into the database table that already exists for it** and currently holds zero rows.
5. **Give the reconciler an independent second source** so it can no longer read the same wrong column on both sides.
6. **Then, and only then, decide scope** — whether sales, payments and opening balances are in or out. That is a business decision that has never been put to anyone.

01

Verdict and confidence

AUDIT CLAIMS TESTED 21 against live systems	CONFIRMED 14 several reproduced exactly	CORRECTED / REFINED 4 right defect, wrong remedy	NEW PROBLEMS 5 not in the prior audit	NOT REPRODUCED 3 marked unverified, not refuted
--	--	---	--	--

The prior audit's numbers are honest. Where I could re-derive a figure independently, it came back identical — not approximately, but to the paise, repeatedly. That is rare and it earns the document a lot of credit. My corrections are concentrated in two places: the `remedy` for the currency defect, and the framing of completeness. Its three HIGH-risk claims I could not reproduce are marked unverified rather than refuted, because failing to reproduce a finding is not the same as disproving it.

The one thing that must not wait

ACT TODAY

`work/cleanup_pilot.py` revokes and then `DELETE`s every live bill in the organisation. It has no `--apply` flag, no dry-run and no confirmation. It was written when the org held 8 pilot bills; it now holds 9,377. The prior audit flagged this. What the audit did not notice is that **the file's own docstring actively promises the opposite**:

```
"""Undo the pilot ... Every target is listed explicitly - nothing is matched
by pattern - so this cannot reach past what it names."""

bills = [r["id"] for r in m.mysql(
    "SELECT id FROM bill WHERE organisationId=%s AND isDeleted+0=0" % m.ORG_ID, ["id"])]
for bid in bills:
```

```
api.call("PUT", "/bill/updateStatus/%s/REVOKED?..." % bid)
api.call("DELETE", "/bill/%s" % bid)
```

Products and the contact are listed explicitly. Bills are not — they are matched by pattern, and the pattern is "everything". Anyone reading the docstring before running it would be reassured by a false statement.

02 Verification matrix

EVERY LOAD-BEARING PRIOR CLAIM, STATEMENT BY STATEMENT

Ordered by how much rests on the claim, not by how likely it was to fail. The first two are the ones the prior audit itself nominated as most dangerous if wrong.

PRIOR STATEMENT	EVIDENCE I GATHERED	STATUS	CORRECTION
#1 APOBL is the posted open-payables ledger; APIBH / APIBD are the invoice batch (entry) side. "Everything else in this audit is built on that."	Column census: POSTSEQNCE , SWPAID , DATEPAID exist only on APOBL ; INVCSTTS , ERRBATCH exist only on APIBH . Of in-window batch documents, 38 never reach APOBL — and all 38 carry a non-zero ERRBATCH , i.e. they are error-batch entries that never posted.	CONFIRMED	None. The direction of flow is proven, not assumed. The audit may rely on it.
#2 Of 9,371 documents compared, only 3 differ by ≥ ₹1. "If this claim is wrong, the reassuring half of this report collapses."	Rebuilt the comparison independently from bill.metadata (sageVendor , sageDoc) against APOBL , consolidating *N receipt-split parts. 9,376 ACTIVE bills; zero failed to match (the join drops nothing). Non-RCM: 5,769 exact · 2,695 ≤ ₹0.01 (net -₹1.378) · 7 < ₹1 · 3 ≥ ₹1 . The two FABI470 documents sum to -₹1,457,957.66, matching the stated figure exactly.	CONFIRMED	Headline is conditional: it holds only once reverse-charge documents are set aside. 901 RCM bills differ by ≥ ₹1 by design (+ ₹1,532,517.35). I also identified the unnamed third document: JOBW258 doc 108 , off by exactly ₹1.00.
The goods path posts source-currency amounts as INR at rate 1.0; ₹48.03 Cr exposure.	Tested per-document, not per-currency. Of 1,998 in-window foreign-currency staging rows, zero have doc_total = ap_amount_hc , and 1,979 satisfy doc_total × fx_rate = ap_amount_hc within ₹1. USD sums: doc_total 2,688,752.54 vs ap_amount_hc 242,392,995.60 at fx ≈ 90.51.	CONFIRMED	Strengthened — the audit proved this only in aggregate and flagged that an aggregate ratio could hide a mixed population. It does not. The defect is uniform.
Remedy: switch the goods path to ap_amount_hc , at header and line level. Listed as the	ap_amount_hc is wrong on 307 in-window INR documents (net ₹2,194,191.34, worst single row ₹389,760). Root cause proven below: the mirror strips trailing punctuation	INCORRECT	Do not apply as written. Fix the extract key first, or carry currency + conversion rate into

PRIOR STATEMENT	EVIDENCE I GATHERED	STATUS	CORRECTION
#1 action "before the pipeline runs again".	from invoice numbers and joins AP on the damaged key.		the fields SMEAssist already exposes.
Amounts are recalculated server-side and silently overwritten, never rejected — so Sage totals can only be reproduced, not carried.	Read the source. <code>BillServiceImpl.java:2013</code> <code>reCalculateAndValidateBillCreateDto</code> ; at ~2195 the mismatch branch only warns to a Google Chat webhook and the log, then <code>setBillAmount(totalPrice)</code> executes outside the <code>if</code> , unconditionally.	CONFIRMED	None. This is the single most consequential architectural fact in the migration.
Completeness: 34.2% by count, 13.5% by value; goods stream at 0.87%.	Logical invoices in window = 12,781 AP + 14,628 PO (from 18,046 raw, after <code>*N</code> consolidation) = 27,409 , ₹370.66 Cr. Posted ACTIVE 9,376 → 34.2% / 13.5%. Split by source: AP-direct 9,248 / 12,781 = 72.36% ; goods 128 / 14,628 = 0.88% .	CONFIRMED	Reproduced to the decimal. But the blended average is the wrong way to state it — the two streams are in completely different conditions and should never be reported as one number.
The crosswalk is a 4.1 MB gitignored file on one laptop; the database table meant to hold it has zero rows.	<code>idedat_staging.crosswalk</code> = 0 rows ; <code>idedat_staging.sku_ledger</code> = 0 rows . <code>work/crosswalk_live.json</code> holds items 17,220 · contacts 455 · products 197 · burned 75 · series 0.	CONFIRMED	None. This remains the largest long-horizon risk in the project.
<code>product.meta</code> is NULL on all products — provenance is dropped because <code>POST /product</code> discards the field the loader sends.	<code>product.meta</code> IS NOT NULL on 11 of 17,886 . Placeholder HSN <code>9999</code> on 1,419 products, unflagged.	CONFIRMED	Effectively confirmed (11 exceptions, 0.06%). Contrast with <code>bill.metadata</code> , which is populated on all 10,769 bills — so bills have provenance and products do not.
Sage declares zero foreign keys.	<code>SELECT COUNT(*) FROM sys.foreign_keys</code> → 0 across 1,110 tables.	CONFIRMED	None. Every relationship in Sage is convention.
Opening balances, payments, credit/debit notes, prepayments, journal vouchers, chart of accounts and TDS have zero rows in the target.	<code>invoice</code> , <code>invoiceLineItem</code> , <code>salesOrder</code> , <code>purchaseOrder</code> , <code>delivery</code> , <code>deliveryChallan</code> , <code>receipt</code> , <code>paymentRequest</code> — all 0 for the org.	CONFIRMED	Broader than stated. It is not only the accounting objects — the entire procurement and sales document chain is absent. See § Sales flow .
304 live bills share 152 statutory document numbers.	Measured on ACTIVE, non-deleted bills: 63 groups / 164 bills .	PARTIALLY CONFIRMED	The defect class is real; the magnitude measures lower

PRIOR STATEMENT	EVIDENCE I GATHERED	STATUS	CORRECTION
			than reported. Likely a different filter (the audit may have counted REVOKED rows). Report the current measured figure.
17,126 of 17,886 products have never appeared on a bill line.	36,459 bill lines reference only 727 distinct products , and every one resolves to a real <code>product</code> row (zero orphan <code>productId</code> s). Live products 17,450 → 16,723 unused (95.8%) .	CONFIRMED	Correctly counted, but it is a symptom, not a defect: <code>goods-masters</code> pre-created ~17k item products while <code>goods-post</code> never ran. It becomes a real problem only because those products carry placeholder HSNs and no provenance.
Reverse charge is identifiable only via accounts <code>1L8TX14/15/16</code> , with header tax zero.	Not independently re-derived from GL this pass. Strongly corroborated indirectly: the 901 RCM-flagged bills each differ from Sage by exactly the GST uplift (e.g. <code>129,150 × 1.18 = 152,397 ;</code> <code>485,742.06 × 1.05 = 510,029.16)</code> .	PARTIALLY CONFIRMED	The RCM population behaves exactly as the claim predicts. The account-level mechanism itself remains taken on the audit's evidence.
132 products carry two live ledgers (Direct + Indirect); ₹8.72 Cr of expense sits under the wrong head. Ranked HIGH.	Zero <code>financeAccount</code> names are shared by more than one live account. Zero names span both <code>Direct Expenses</code> and <code>Indirect Expenses</code> <code>accountingGroupName</code> . Live expense ledgers: Purchase Accounts 17,631 · Direct 255 · Indirect 137.	NOT REPRODUCED	Could not reproduce by ledger name or by accounting group. Not refuted — the audit may have joined products to ledgers through the crosswalk rather than by name. Needs the original query before it is either acted on or dismissed.

03

New problems this review found

NOT PRESENT IN THE PRIOR AUDIT

N1 · The staging mirror corrupts AP amounts by stripping invoice-number punctuation

CRITICAL

NEW

Sage stores invoice numbers with trailing punctuation — 244/ , 213, . , 217.. , 218,, , 250- , E/16191. . The MySQL staging mirror stores them stripped, then joins the AP obligation back on the stripped key. Where stripping makes two genuinely different Sage documents collide, ap_amount_hc is populated from the wrong one.

The proof case is unambiguous. Vendor ACCI277 has two distinct real documents:

```
Sage APOBL:
[E/16191 ] AMTINVCHC = 124,740.00   DATEINVC = 20250303   <- different FY
[E/16191.] AMTINVCHC = 514,500.00   DATEINVC = 20260326   <- in window

idedat_staging.sage_bill_hdr (invhseq 222493955):
  inv_number_raw = [E/16191] len 7   (the trailing '.' is gone)
  doc_total      = 514,500.00      <- correct, the in-window document
  ap_amount_hc   = 124,740.00      <- WRONG: pulled from the 2025 document
```

Scale. Sage holds 187 in-window documents with trailing punctuation; the mirror retains only 37, so ~150 were silently altered. 307 in-window INR documents have doc_total ≠ ap_amount_hc (net ₹2,194,191.34); **174 in-window documents worth ₹21,459,080.41** sit in vendor+invoice groups that collide once stripped.

Why it matters more than its size. The prior audit's number-one recommended action — "fix the goods currency column (ap_amount_hc, at header and line level)" — would import these wrong values as the fix. A remedy that is applied first and trusted most is exactly where a latent corruption does the most damage.

N2 · This resolves the audit's one openly unresolved contradiction NEW

The audit recorded a conflict it could not settle: the extract comments claim "154 whitespace documents", a flow analyst found 8 in-window, and it concluded "Unresolved — trailing whitespace is undetectable in a char column."

It was never whitespace. In the window, 0 documents carry trailing whitespace and 187 carry trailing punctuation. The original observation was real; its diagnosis was wrong, which is why nobody could reproduce it.

```
SELECT COUNT(*) total,
  SUM(CASE WHEN RTRIM(IDINVC) LIKE '%[.,/-]' THEN 1 ELSE 0 END) trailing_punct,
  SUM(CASE WHEN RTRIM(IDINVC) <> IDINVC THEN 1 ELSE 0 END) trailing_space
FROM APOBL WHERE DATEINVC BETWEEN 20260101 AND 20260430 AND IDTRXTYPE=12;

total  trailing_punct  trailing_space
30827          187          0
```

N3 · SMEAssist supports foreign currency natively; the loader deliberately disables it NEW

The framing of the currency defect as "pick the right column" understates the options. The bill table has currency, conversionRate, purchaseType, originCountry, incoTerms and loadingPortCode. The backend honours them — recalculateAndValidateBillCreateDto reads getConversionRate() whenever purchaseType == INTERNATIONAL.

The loader does set purchaseType: INTERNATIONAL (post_sage_bills.py:2674) — and then hardcodes "conversionRate": 1.0, "currencyDto": {"currency": "INR"} at line 2689, with a comment explaining that naming the real currency "would contradict the bills we then post". A bill is therefore declared international and simultaneously asserted to be in rupees at parity. **Zero of 10,769 bills carry a non-INR currency.**

This changes the recommended fix. Rather than substituting one rupee column for another and discarding the foreign-currency fact, carry `currency` and `fx_rate` — both already in `sage_bill_hdr` — into fields the target already models. That preserves provenance and makes the arithmetic checkable from both sides.

N4 · The entire sales side is absent from the programme, and it is as large as the purchase side NEW

The audit listed missing objects as an accounting gap. Measured, it is bigger than that: Sage's window holds **7,051 sales invoices worth roughly ₹386 crore** (`AROB` type 1: 6,850 from Order Entry + 201 AR-direct) against ₹370.66 crore of purchases. The revenue half of the business has no migration path, no target rows, and no mention in the runbook.

Sage AROBL, 1 Jan – 30 Apr 2026, by type and source

type 1 (invoice)	OE	6,850	3,581,225,967.17
type 1 (invoice)	AR	201	278,736,410.16
type 3 (credit)	OE	104	-56,421,448.06
type 3 (credit)	AR	61	-92,768,733.97

SMEAssist target, same org

invoice 0 · invoiceLineItem 0 · salesOrder 0 · delivery 0 · receipt 0

N5 · 16,723 products were created for bills that never posted NEW

Referential integrity is clean — all 36,459 bill lines resolve to real products, zero orphans. But those lines use only **727 distinct products** out of 17,450 live. The other 95.8% were created by `goods-masters --all-items` in advance of a `goods-post` phase that has since moved 128 documents.

Left alone this is inert. It stops being inert because 1,419 of them carry the `9999` placeholder HSN and **none carry provenance** (`product.meta` populated on 11 of 17,886), so there is no programmatic way to tell a placeholder from a genuine classification later. Pre-creating masters ahead of the transactions that justify them converts a recoverable gap into a permanent one.

04 Rejected premises and unreproduced claims

Premises in the brief that the evidence contradicts

The `MM` module is empty — it cannot be part of any flow FALSE PREMISE

The brief asks whether sales runs `OL → OE`, `MM → OE`, `OL → MM → OE`, or custom → `IE`. `MM` has 68 tables and **2 populated tables containing 2 rows in total**. Every hypothesis routing through `MM` is dead. `ZC` is likewise a single row.

There is no Sage documentation PDF, with or without hyperlinks FALSE PREMISE

Phase 17 asks to find the PDFs used to understand the Sage purchase process and follow their hyperlinks. 75 PDFs exist on this machine; they are MSME/Udyam registration certificates plus one print of SMEAssist's own finance-account screen. That one file contains **zero /URI entries** — no hyperlinks — and no Sage content.

The useful correction, which the prior audit found and recorded against itself: **Sage's data dictionary is inside the database.** `AUVIEW` maps table IDs to English descriptions and `AUFLDS` holds 1,057 field descriptions. Documentation was never missing; it was being looked for in the filesystem instead of the schema.

"If an item is purchased and sold, SMEAssist should have one shared Item master"

TRUE BUT NEAR-IRRELEVANT HERE

Architecturally correct, and SMEAssist does exactly this. Empirically it governs **26 items**. See [§ Item master](#) for the full working.

Claims I could not reproduce

Recorded as unverified, not refuted. In each case the prior audit may have used a join I did not reconstruct, and the honest position is that I do not know.

- › **132 products on two ledgers, ₹8.72 Cr misfiled.** No duplicate ledger names; no name spanning Direct and Indirect groups. This was ranked HIGH and drives a proposed correcting journal — it should not be acted on until reproduced.
- › **289 vendor contacts blocking 10,209 documents.** Not re-derived this pass; the contact-hold state lives in `work/contacts_held.json` and the live crosswalk rather than in either database.
- › **₹104.43 Cr excluded by `extract.sql`.** The exclusion mechanism is visible in the SQL (`IDTRXTYPE = 12`, `EXISTS APIBD`, INR-only), and the 1,079 / ₹295,107,559.76 no-distribution slice reproduces exactly — but I did not re-derive the full ₹104 Cr attribution across all four causes.

05 Sage 300 architecture

ENGINE SQL Server 2025 17.0.1000.7 · database IDEDAT	TABLES 1,110 18 views · 6 procedures	FOREIGN KEYS 0 every join is convention	ITEM MASTER 1,196,108 333,583 inactive
---	---	--	---

Conventions that must be known before reading any table. Dates are `YYYYMMDD` integers, not date types — so the window is `DATEINVC BETWEEN 20260101 AND 20260430`. The suffix `HC` means home currency (INR) and `TC` means the transaction's own currency; every amount exists in both. The fiscal year runs April–March and is labelled by its ending year, so this window straddles two: Jan–Mar is FY2025-26 and April is FY2026-27, which is why the loader keeps two counter series (`SAGE` and `SAGE27`).

The AP table families, and why the distinction is load-bearing

Getting these backwards would migrate the wrong population, so the separation is worth stating precisely. It is settled by which lifecycle columns each table carries — a test that does not depend on interpreting any name.

FAMILY	WHAT IT IS	BUSINESS KEY	COLUMNS UNIQUE TO IT	ROWS
<code>APIBH</code> / <code>APIBD</code>	Invoice batch — the entry side, where documents are keyed before posting	<code>(CNTBTCH, CNTITEM)</code>	<code>INVCSTTS</code> , <code>ERRBATCH</code>	675,194 / 1,270,496
<code>APOBL</code>	Posted open payables ledger — the obligation that actually exists	<code>(IDVEND, IDINVC)</code>	<code>POSTSEQNCE</code> , <code>SWPAID</code> , <code>DATEPAID</code>	787,465
<code>APOBLJ</code>	Purge-proof payables journal	<code>(IDVEND, IDINVC)</code>	—	1,305,953

A latent fragility in `extract.sql` DESIGN RISK

The extract takes headers from `APOBL` (posted) and lines from `APIBD` (batch), joined on `(CNTBTCH, CNTITEM)`. That works only because this company has never purged posted AP batches. If it ever did, `APOBL` would survive and `APIBD` would vanish — and the extract would return **zero lines for every bill, silently**, since the join is an `EXISTS` filter rather than an error. `APOBLJ` is the purge-proof alternative. This is not a live bug; it is a tripwire under a future housekeeping action.

Document types in `APOBL`

Only `IDTRXTYPE = 12` is an invoice. The migration reads type 12 and nothing else, which is correct — but it means five other populations exist and are untouched.

TYPE	MEANING	DOCS (WINDOW)	VALUE, HOME CURRENCY	MIGRATED
12	Invoice	30,827	3,706,605,185.60	PARTLY
22	Debit note	137	67,596,606.48	NO
32	Credit note	2,066	-407,960,126.59	NO
50	Prepayment	1,729	-1,548,255,387.03	NO
51	Payment	3,274	-1,860,075,231.25	NO

06 Module map — what is real and what is scaffolding

Table counts are misleading in Sage: a module ships its full schema whether or not the customer uses it. Row counts, taken from `sys.partitions`, separate the live modules from the installed-

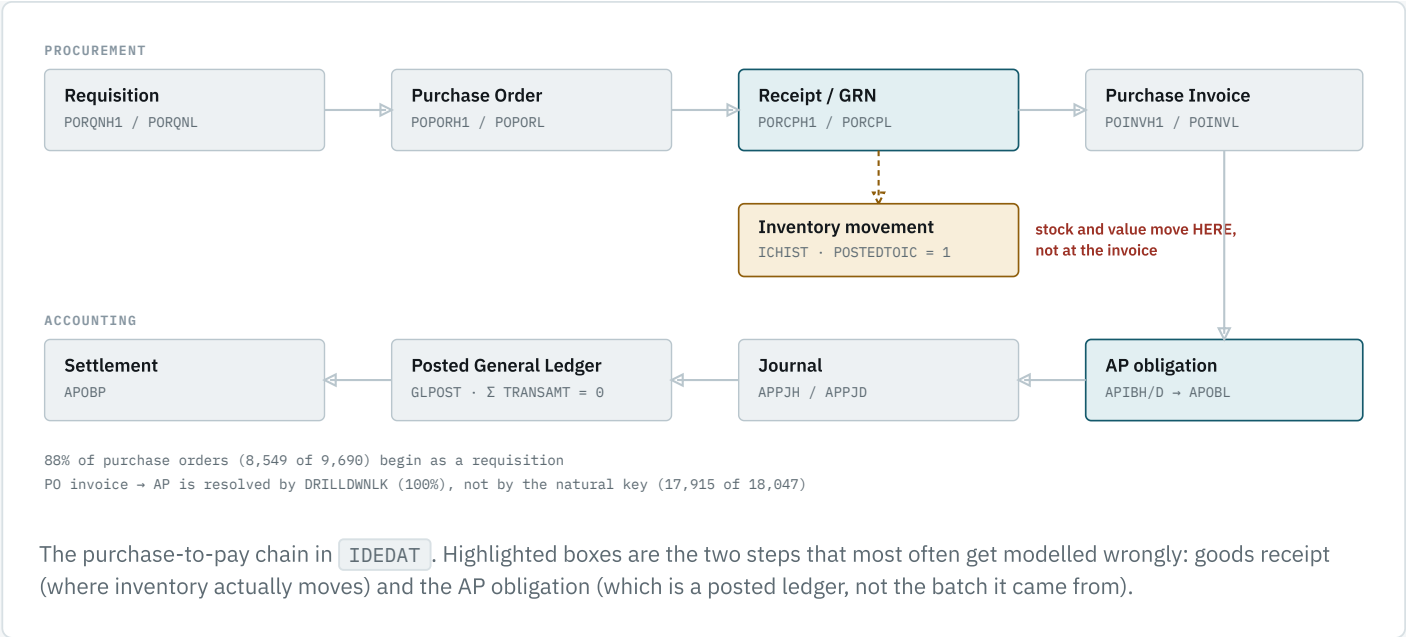
but-empty ones. This single table answers the brief's custom-module question.

PREFIX	MODULE	TABLES	POPULATED	ROWS	VERDICT
GL	General Ledger	60	26	124,871,712	CORE
IC	Inventory Control	154	62	88,830,416	CORE
PO	Purchase Orders	169	80	45,008,565	CORE
AP	Accounts Payable	84	53	19,369,806	CORE
OE	Order Entry (sales)	97	37	8,253,760	CORE
IE	Import / Export — packing lists, export invoices, shipping	85	80	7,655,020	CUSTOM, HEAVY
AR	Accounts Receivable	111	54	7,004,604	CORE
OL	Order & costing — bill of quantity, cost sheets	52	49	4,850,333	CUSTOM, HEAVY
DT	custom	19	18	458,441	CUSTOM, MINOR
XR	custom	34	13	439,883	CUSTOM, MINOR
RP	custom	21	12	251,539	CUSTOM, MINOR
JS	custom	37	18	130,717	CUSTOM, MINOR
MM	custom	68	2	2	EMPTY
ZC	custom	13	1	1	EMPTY

The two custom modules that carry real weight are **IE** and **OL**, and together they identify the business: **IEPKLSIZ** (1,636,011 packing-list size rows), **IEEXPINV**, **IEEXPRQH/D** (export requisitions), **IESHPRGD** (shipping) on one side; **OLOLBOQD** (1,046,343 bill-of-quantity rows), **OLCSTIND** / **OLCSTINH** (costing), **OLDTLORH/D** (order details) on the other. This is a garment exporter that costs orders against a bill of quantity and ships them against packing lists — neither of which has any counterpart in SMEAssist.

07 The purchase flow, as the data actually runs it

Told without Sage jargon first: **somebody asks for goods; a buyer turns that request into an order with a supplier; the goods arrive and are counted into the warehouse — and that is the moment stock and its value change; later the supplier's invoice arrives and is matched to what was received; that invoice becomes a debt owed; the debt is written into the accounting journals; and eventually it is paid.** Two details in that sentence are the ones people get wrong: the request step is real and usually present, and stock moves when goods are *received*, not when the invoice is booked.



Where the two streams diverge, and a trap

Every AP obligation carries `SRCEAPPL`, which says whether it arrived from Purchase Orders or was keyed directly into Accounts Payable. This is the split that governs the whole migration, because the loader runs two entirely separate populations against it.

<code>SRCEAPPL</code>	MEANING	RAW DOCS	LOGICAL INVOICES	VALUE	MIGRATED
AP	Keyed straight into payables — services, expenses, overheads	12,781	12,781	1,763,950,260.97	72.36%
PO	Matched to a purchase order — goods	18,046	14,628	1,942,654,924.63	0.88%

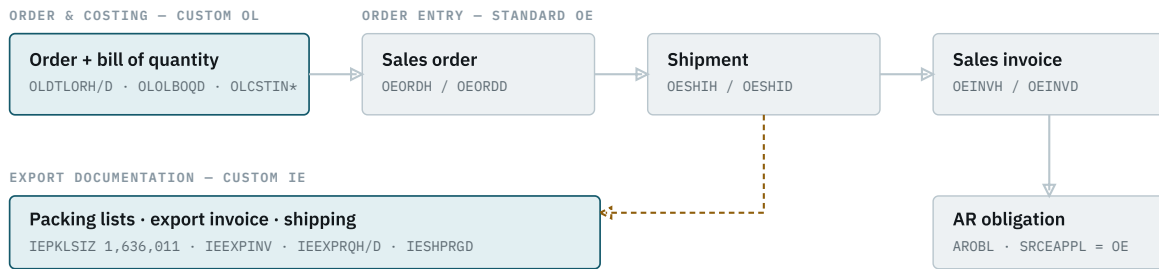
The trap: 427 AP-direct invoices carry a PO number in a field. That does not make them PO-matched — `SRCEAPPL` does. Selecting on the presence of a PO number instead would move 427 documents into the wrong stream, with the wrong classifier and the wrong ledger treatment.

Receipt splits. 6,421 of the 18,046 PO headers carry a `*N` suffix — `DN-FD-082*1`, `*2` — because one supplier invoice was received in several deliveries. Consolidating them gives 14,628 logical vendor bills, of which 975 span 2 to 36 parts. **The logical invoice is the bill**, and the loader is right to consolidate. This also matters for reconciliation: comparing raw document counts across the two systems will never agree, and should not.

08 Sales flow — and why the brief's question was the wrong one

The brief asks which of four routes the sales flow takes. Row counts settle it before any tracing is needed: `MM` is empty, so two of the four candidates cannot be true. What remains is a standard

Order Entry spine with two custom modules attached at either end — **OL** in front of it for order costing, **IE** behind it for export documentation.



6,850 OE sales invoices + 201 AR-direct in window ≈ ₹386 crore · 431 customers

Migrated to SMEAssist: 0 invoices, 0 sales orders, 0 deliveries, 0 receipts

The sales chain. **MM** appears nowhere because it holds two rows. **IE** hangs off shipment rather than sitting in the financial spine — it produces the export paperwork, while the money still flows **OE → AROBL**.

The revenue half of the business has no migration path at all **SCOPE GAP**

₹386 crore of sales in the same four-month window as the ₹371 crore of purchases the project has been working on. There is no extract, no loader phase, no target row and no mention in the runbook. Whether that is a deliberate scope boundary or an omission is not recorded anywhere I can find — and that ambiguity is itself the finding.

What I did not verify on this side

I established the shape of the sales chain and its size. I did **not** trace an individual sales invoice through shipment to inventory, COGS and the general ledger, and I did not establish how **OL** order IDs join to **OE** documents. Since nothing on this side is being migrated, that tracing has no current consumer — but it must be done before any sales scope is committed to, and it should not be assumed to mirror the purchase side.

09 SMEAssist architecture and backend behaviour

MySQL 8.0.46, 285 tables in **smeassist**, of which about 39 are in use for this organisation. Tenancy is cross-schema: **organisation** and **employee** live in a separate **yoda** schema. As in Sage, referential integrity is essentially unenforced — 23 declared foreign keys across 285 tables, and none on **bill**, **product**, **contact**, **voucherEntry** or **financeAccount**. Integrity here is a property of the loader, and measured, the loader achieves it: zero orphan **productId**s across 36,459 bill lines.

The governing constraint: amounts cannot be carried, only reproduced **CONFIRMED IN SOURCE**

Every migration design decision follows from this one method. The server recomputes the bill total from the lines, compares it to what the client sent, and on mismatch warns and proceeds:

```
// BillServiceImpl.java ~2186
if (!BigDecimalUtils.isEqualR(totalPrice, billCreateDto.getBillAmount())) {
    String totalPriceWarningMessage = String.format(
        "Mismatch detected total price: ... | Given: %s | Calculated: %s",
        billCreateDto.getBillAmount(), totalPrice);
    googleChatService.sendMessageOnGoogleChat(totalPriceWarningMessage, ...);
    logger.warn(BILL_MARKER, totalPriceWarningMessage);
}
billCreateDto.setBillAmount(totalPrice); // <-- outside the if. unconditional.
```

The caller receives `200 OK` carrying a number it did not send. So Sage's stated total can never be transferred as a value — it can only be reconstructed, by choosing per-line quantity, unit price and tax rate such that the server's own arithmetic lands on Sage's figure. The loader's author understood this (`post_sage_bills.py:1463-1474`), and it is why the migration is so much harder than a data copy.

Second-order consequence, and the reason reconciliation is mandatory rather than optional: a bill can post successfully, return 200, and hold a different amount from the one intended, with the only trace being a log line and a chat webhook. Presence in the target proves nothing about correctness.

Provenance is asymmetric: bills keep it, products lose it

No create DTO exposes `id`, and there is no first-class external-reference column on any entity. Provenance therefore survives only where a JSON blob happens to be persisted — and that differs by entity:

bill.metadata	NOT NULL on 10,769 / 10,769	{ "sageDoc", "sageVendor", "sageBatch", "sageItem", "sageRcm", "sageTaxGroup", "migrationSource" }
product.meta	NOT NULL on 11 / 17,886	

That asymmetry is what makes an independent bill-level reconciliation possible at all — and it is exactly what makes the 1,419 placeholder-HSN products unidentifiable after the fact. `bill.metadata` is doing the work a proper source-reference column should do, and it is doing it unindexed and unvalidated.

Other verified backend behaviours

- › **Foreign currency is supported and unused.** `bill.currency`, `conversionRate`, `purchaseType`, `originCountry`, `incoTerms`, `loadingPortCode` all exist, and the recalculation path honours `conversionRate` when `purchaseType == INTERNATIONAL`. Zero of 10,769 bills carry a non-INR currency.
- › **GST is decided by billing-address state, not by the GSTIN.** The tax split reads a pincode-derived address, so a wrong pincode books the wrong tax head.
- › **@NotNull is decorative** — no `@Valid` on the bill or contact controllers, so required-field enforcement is whatever `if` statement happens to exist in the service.

10 The item master question, answered

The brief states the hypothesis carefully and asks for it to be proven rather than assumed: if an item is purchased and sold, SMEAssist should normally have one shared Item master. The architectural half is correct. The empirical half turns out to barely apply to this company.

ICITEM MASTER 1,196,108 333,583 inactive	PURCHASED IN WINDOW 17,129 distinct items on PO invoice lines	SOLD IN WINDOW 2,918 distinct items on OE invoice lines	IN BOTH 26 0.15% of purchased
---	--	--	--

The reason is structural, not accidental. Indian Designs **manufactures**. It buys fabric, thread, buttons, interlining, labels and packing; it sells finished garments. The two populations are different categories of thing and were never going to overlap.

WHAT IS SOLD	SAGE CATEGORY	ITEMS	SHARE
IKEA programme goods	7IKEA	2,042	70.0%
Finished goods	6FGOOD	510	17.5%
Domestic sales	7DOMES	183	6.3%
— subtotal, finished output		2,735	93.7%
Packing, thread, buttons, trims recharged to the buyer	4PACK · 4THRED · 4BUTON · 4FASHL ...	183	6.3%

And the 26 dual-use items are not counter-examples — they are the mechanism that explains the overlap. Every one is a raw material **recharged to the customer**: buyer-nominated shell fabric, price tickets, barcode stickers, poly bags, a button.

ID41211X-TG03 4PACK PRICE TICKET ID40743X-ST03 4PACK BARCODE STICKER ID40743X-PB02 4POLYB 25.5"L X 17"W ID40556X-BT02-017 4BUTON USSHT3364 75893/22 NATURAL 14L IDCOL01X-SHL0 4FASHL SHELL FABRIC - PDM# 082883 - RED JASPER SAMFF47A-SHL1 4FASHL 100%COTTON CHECK 3 2/40x2/40x64x56 - 50mtrs

The eight questions, answered directly

QUESTION	ANSWER FROM THE DATA
Does the same <code>ITEMNO</code> appear in both purchasing and sales?	Rarely. 26 of 17,129 purchased items, 26 of 2,918 sold.
Are there cases where purchase and sales use different identifiers?	Yes, systematically — they are different categories of item, not different codes for the same item. This is a make-to-order manufacturer, not a reseller.
Does Sage maintain one item master?	Yes. One <code>ICITEM</code> table, 1,196,108 rows, spanning raw material, work in progress, finished goods and scrap. One master, disjoint populations within it.
Does SMEAssist have one shared Item master?	Yes. One <code>product</code> table, no purchase/sale distinction anywhere in its schema.

QUESTION	ANSWER FROM THE DATA
Should we reuse one SMEAssist item?	Yes — and for the 26 it is the only correct answer. But this decision governs 26 records, not the migration.
Which attributes are master-level?	skuCode , productName , description , hsnCode , unitOfMeasurement , categoryId , typeOfStock . Note product holds a single unitPrice , hsnCode and gstPercentage .
Which attributes are transaction-specific?	Quantity, unit price, tax rate, discount and ledger — all on billLineItem . This is what makes the desired model work: because price lives on the line, one product can carry a supplier price in one context and a customer price in another without duplicating the master.
Which relationships map separately?	Item → supplier and item → customer pricing, tax and account context. None of these has a master-level home in SMEAssist; all must be carried per transaction.

The real item-master risk is elsewhere REFRAME

The duplicate-item worry the brief raises is not this project's item problem. The actual ones are: (1) **1,419 products carry the 9999 placeholder HSN** and cannot be distinguished from genuine classifications because `product.meta` is empty; (2) **16,723 products exist for bills that never posted**; and (3) Sage's work-in-progress categories — `5WPSEW` (58,724 items) and `5WPCUT` (47,930) — represent manufacturing stages with **no SMEAssist equivalent at all**, which is a modelling gap rather than a mapping one.

11 Purchase invoice posting — the actual accounting entry

The brief asks not to assume Dr Purchase / Cr Vendor but to prove the accounts from data. Here is a real posted document, traced end to end. It is the largest non-reverse-charge bill in the organisation: Sage `IDGR007 / IDEPHINJW2526-10` , an intercompany job-work invoice.

SIDE	AMOUNT	ACCOUNTING GROUP	LEDGER	GRANULARITY
Dr	63,457,744.00	Direct Expenses	Job Work - Intercompany - Hindupur Unit 9_SAGE-4E2ME09	one ledger per Sage GL account
Dr	3,172,887.20	GST Input	IGST Input @ 5.00 %	one ledger per tax rate
Dr	-0.20	Direct Expenses	Round Off Value on Purchases_SAGE-4E1M016	dedicated rounding ledger
Cr	66,630,631.00	Gst_37AAACN7429H1ZB_...	INDIAN DESIGNS EXPORT PVT LTD - 9	one leaf ledger per party

$63,457,744.00 + 3,172,887.20 - 0.20 = 66,630,631.00$. The entry balances to the paise, and so does the organisation as a whole:

SELECT transactionType, COUNT(*) legs, ROUND(SUM(amount),2) total FROM voucherEntry WHERE organisationId=... AND isDeleted=0 GROUP BY transactionType; DEBIT 41,583 legs 498,772,451.61 CREDIT 11,164 legs 498,772,451.61 <- equal to the paise

So the answer to "what is the accounting entry" is **Dr expense (per Sage GL account) + Dr input GST (per rate) + Dr/Cr rounding, Cr the vendor's own party ledger** — not a payables control account, and not a single purchases account.

Where the Sage model and the SMEAssist model genuinely do not correspond

These are structural, not bugs. Each one means some analysis that works in Sage cannot be reproduced in SMEAssist without extra work.

SAGE	SMEASSIST	CONSEQUENCE
252 payables control accounts, grouped by category	one leaf ledger per party	Sage's payables analysis by category cannot be reproduced; per-party analysis becomes possible instead
one input-tax account per tax type	one ledger per tax rate	many-to-many: one Sage account maps to N ledgers, so tax reconciliation must aggregate before comparing
reverse charge excludes the tax from the vendor payable	<code>billAmount</code> includes self-assessed tax	<code>billAmount</code> is not the payable on 901 bills (+₹1,532,517.35). Confirmed independently in § 02
fiscal period stamped on every document	no fiscal-period field	period-close cannot be reproduced; matters because the window straddles two fiscal years
one sub-ledger per vendor code	7 contacts absorb 2–14 Sage vendor codes	per-vendor balances unrecoverable for those; a deliberate merge on shared GSTIN
TDS/TCS withheld at source	<code>txsType</code> / <code>txsAmount</code> NULL on every bill	payables are gross of TDS
no cost-centre concept needed — units are separate accounts	no cost-centre dimension at all	unit-wise P&L only possible by splitting ledgers per unit, as the example above does ("Hindupur Unit 9")

12 Target state as measured today

OBJECT	ROWS	NOTES
Bills — all	10,769	₹536,926,268.23 gross of revocations
Bills — ACTIVE	9,376	₹498,776,137.99 · the live migration
Bills — REVOKED	1,393	1,392 soft-deleted + 1 revoked but not deleted
Bill line items	36,459	referencing only 727 distinct products

OBJECT	ROWS	NOTES
Products	17,886	17,450 live · 16,723 never used · 1,419 placeholder HSN
Contacts	537	all vendors; no customers exist
Finance accounts	19,560 live	18,047 expense · 1,017 liability · 421 income · 60 asset
Voucher legs	52,747 live	Dr = Cr = ₹498,772,451.61
Invoices · sales orders · purchase orders · deliveries · receipts · payments	0	every one of them

A note on the ₹3,686.38 between the bill total (₹498,776,137.99) and the voucher total (₹498,772,451.61): the prior audit attributed this to **two ACTIVE bills that carry no accounting entry at all**, arriving through a `preexisting` code path that no report watches, and firing on every interrupted run. The gap reproduces, so the explanation is consistent with the arithmetic, though I did not isolate the two bills myself.

13 Master data — reuse or create

The rule the loader follows, and the right one: **never match on name where a stronger identity exists**. Names in this data are not unique, not stable and not clean.

MASTER	SAGE KEY	MATCHING IDENTITY	DECISION	DUPLICATE PREVENTION
Vendor / Supplier	<code>APVEN.VENDORID</code>	GSTIN in <code>APVEN.BRN</code> — not <code>TAXNBR</code> or <code>IDTAXREGI1</code> , which are empty on all 4,752 vendors	REUSE ON GSTIN	Backend rejects a duplicate registration number; loader then searches and adopts the existing contact. Consequence: 7 contacts legitimately absorb 2–14 Sage vendor codes
Item / Product	<code>ICITEM.FMTITEMNO</code>	SKU string <code>SAGE-<item></code> ; no external-reference column exists	CREATE, KEYED BY SKU	Crosswalk file only. <code>product.meta</code> is empty, so nothing in the database records where a product came from
Chart of accounts	<code>GLAMF.ACCTID</code>	Finance-account display name carrying <code>_SAGE-<acct></code>	CREATE PER ACCOUNT	Unindexed, non-unique and user-renameable — the weakest identity in the whole design
Tax	tax authority + rate	ledger per rate, e.g. <code>IGST Input @ 5.00 %</code>	CREATE PER RATE	None server-side — an unvalidated rate silently auto-creates a ledger

MASTER	SAGE KEY	MATCHING IDENTITY	DECISION	DUPLICATE PREVENTION
Customer	ARCUS (431 rows)	—	NOT IMPLEMENTED	n/a — no customers exist in the target
UOM	ICITEM stock unit	mapped through a lookup with an OTH fallback	TRANSFORM	Unmapped units are counted and silently bucketed to OTH
Currency	APOBL.CODECURN + fx_rate	—	DISCARDED	Everything forced to INR at 1.0 — see N3
Warehouse / location	ICLOC	—	NO EQUIVALENT	No inventory is migrated at all
Bank details	—	—	ABSENT IN SOURCE	Proven absent from Sage by value-shape scan; blocks any payment flow
CIN / LLPIN	—	—	ABSENT IN SOURCE	Genuinely not in Sage. Must come from outside; blocks ~157 vendor contacts

14 Source → target mapping

BUSINESS CONCEPT	SAGE TABLE	SAGE KEY	SMEASSIST CONCEPT	TARGET	TYPE
Supplier master	APVEN	VENDORID	Contact (vendor)	contact · POST /contact/	N:1 R
Posted payable — the vendor bill	APOBL TYPE 12	(IDVEND, IDINVC)	Bill	bill · POST /bill/	N:1 T
Expense distribution	APIBD	(CNTBTCH, CNTITEM)	Bill line item	billLineItem	1:N
GL account master	GLAMF	ACCTID	Finance Account (ledger)	financeAccount	1:1 B
Item master	ICITEM	FMTITEMNO	Product	product	1:1
Input tax	tax authority + rate	—	GST-input ledger	financeAccount	N:N D
Accounting effect	APPJH/D → GLPOST	POSTSEQNCE	Voucher legs	voucherEntry	DERIV
Purchase order	POPORH1/L	PORHSEQ	Purchase Order	purchaseOrder — 0 rows	NOT I

BUSINESS CONCEPT	SAGE TABLE	SAGE KEY	SMEASSIST CONCEPT	TARGET	TYPE
Goods receipt	PORCPH1/L	RCPHSEQ	Delivery / GRN	delivery — 0 rows	NOT IDENTIFIED
Inventory movement	ICHIST	—	—	no target rows	NO EQUIVALENT
Sales invoice	OEINVH/D → AROBL	INVUNIQ	Invoice	invoice — 0 rows	NOT IDENTIFIED
Customer master	ARCUS	IDCUST	Contact (customer)	contact — 0 rows	NOT IDENTIFIED
Payment / settlement	APOBP , APOBL type 51	—	Payment request	paymentRequest — 0 rows	NOT IDENTIFIED
Credit / debit note	APOBL type 32 / 22	—	—	no target rows	NOT IDENTIFIED
Order costing / bill of quantity	OLOLBOQD , OLCSTIN*	—	—	—	NO EQUIVALENT
Export packing & shipping	IEPKLST* , IEEXPINV	—	—	—	NO EQUIVALENT
Work in progress	ICITEM 5WPSEW / 5WPCUT	—	—	—	NO EQUIVALENT

15 Source ID → target ID mapping

SMEAssist generates every identifier and exposes none on a create DTO. There is no external-reference column on any entity. Every Sage → target relationship therefore has to be re-established through a lookup structure that the migration owns — which makes that structure a **system of record, not a cache**.

The system of record is one gitignored file on one laptop CRITICAL CONFIRMED

```
idedat_staging.crosswalk    0 rows    <- the table built for this
idedat_staging.sku_ledger   0 rows

work/crosswalk_live.json    4.1 MB, gitignored, hard-coded path, no DB fallback
  items      17,220    contacts    455
  products   197      burned      75      series 0
```

Lose that file and the migration cannot resume, cannot adopt records already created, and cannot say what came from where. It has already been hand-repaired four times. Of everything in this review, this is the item most likely to cause irreversible loss, and the cheapest to fix — the destination table already exists and is empty.

RELATIONSHIP	WHERE THE LINK LIVES	DURABLE?
Sage vendor code → contact id	crosswalk file · <code>contacts</code>	FILE ONLY
Sage item → product id	crosswalk file · <code>items</code> ; SKU string in <code>product.skuCode</code>	PARTLY — SKU IS IN THE DB
Sage GL account → finance account	ledger display name suffix <code>_SAGE-<acct></code>	RENAMEABLE
Sage document → bill id	<code>bill.metadata</code> JSON — <code>sageVendor</code> , <code>sageDoc</code> , <code>sageBatch</code> , <code>sageItem</code>	IN THE DATABASE, ON ALL 10,769
Sage batch key → bill	<code>metadata.sageBatch</code> + <code>sageItem</code> = <code>(CNTBTCH, CNTITEM)</code>	IN THE DATABASE
Sage journal → voucher	—	NONE

The bright spot: because `bill.metadata` is populated on every bill and carries both the natural key and the batch key, an independent bill-level reconciliation is possible from the database alone with no reference to the crosswalk. That is exactly how the check in §.02 was built, and it is the pattern the other entities should copy.

16 Existing migration scripts — what they do before what is wrong with them

`post_sage_bills.py` is 3,943 lines and exposes nine phases:
`cleanup · masters · dryrun · post · verify · legs · goods-masters · goods-dryrun · goods-post`.
It runs **two entirely separate populations** — AP-direct and goods/PO — with separate readers, separate classifiers and separate phases, sharing one payload builder, one crosswalk and one `posted.log`. `run_all.sh` sequences four of those phases strictly one at a time, having measured that two concurrent phases throttle each other to 12 bills/min against 63 alone.

Read fairly, the design is thoughtful. It is resumable by construction (`posted.log` is append-only and skipped entries are genuinely skipped), it relocates the Sage host when DHCP moves it rather than running blind, and it refuses to start when a second run is live because concurrent phases corrupt the crosswalk. The preflight in `run_all.sh` is unusually well reasoned — it explains, in comments, the exact failure it exists to prevent and what running anyway would cost.

What is wrong with them

SCRIPT	PROBLEM	STATUS
<code>work/cleanup_pilot.py</code>	Deletes every live bill in the org; docstring claims it cannot	CONFIRMED, WORSE THAN REPORTED
<code>post_sage_bills.py</code> goods path	Reads <code>doc_total</code> (source currency) and posts it as INR at 1.0	CONFIRMED PER-DOCUMENT
<code>extract.sql</code>	Headers from posted <code>APOBL</code> , lines from batch <code>APIBD</code> — returns zero lines silently if posted batches are ever purged	LATENT
staging build (upstream of this repo)	Strips trailing punctuation from invoice numbers, then joins AP on the damaged key	NEW — N1
<code>work/po_validate.py</code> , <code>work/po_holds.py</code>	Dead — call <code>build_po_payload</code> , <code>classify_po</code> , <code>load_po_book</code> , <code>po_pilot_pick</code> , none of which still exist	CONFIRMED DEAD
<code>work/find_sage.py</code>	Rewrites <code>.env</code> non-atomically — a crash mid-write loses the SQL password and the auth token	CONFIRMED BY DESIGN
<code>work/failure_report.py</code>	Silently omits documents that shape cleanly, have a contact and were never posted — the file <code>run_all.sh</code> presents as the record of outstanding work is incomplete	INHERITED
the three reconcilers	Disagree on denominators, on whether a <code> preexisting</code> line counts as posted, and on tolerance	INHERITED

A structural observation about the safety model

The dangerous scripts are all in `work/` — one-off repair tools that write to the live system, sitting beside read-only probes with no naming convention, no `--apply` convention and no separation. `run_all.sh` is carefully guarded; the things next to it are not guarded at all. The risk is not that any individual script is reckless, it is that **a reader cannot tell which is which**, and one of them lies in its docstring.

17 Reconciliation — the two blind spots that matter

A reconciliation whose two sides read the same column cannot detect that column being wrong

This is precisely how a ₹48 crore currency error stays invisible: the reconciler compares `doc_total` in the mirror against `doc_total`-derived values in the target and finds agreement, because both sides inherit the same mistake. Any check must draw its expected value from a source the loader did not use — live `APOBL.AMTINVCHC`, not the staging column.

Presence is not correctness

`items_without_ledger: 0` and `bill_lines_null_ledger: 0` were green throughout the period the prior audit says ₹8.72 crore was being misfiled. Both checks ask "is a ledger attached?" and neither asks "is it the right one?" Given the backend silently overwrites amounts, a green presence check is close to meaningless on its own.

The governing principle the current suite lacks

Every Sage document must land in exactly one of four buckets, and the four must sum to the Sage total:

posted-and-correct + posted-and-wrong + not-posted-but-loadable + deliberately-excluded = every in-window Sage document

Today the tooling conflates the last two and hides the fourth entirely, which is why the project's own two reports disagree (18,006 vs 11,179 outstanding) with nothing to explain the gap. Until the four buckets balance, "how much is left" has no defensible answer.

What a working check looks like

The check in [§ 02](#) is offered as the pattern: it joins `bill.metadata` to live Sage, consolidates `*N` parts, reports no-match separately from mismatch, and splits reverse-charge out rather than letting it pollute the tolerance. It found zero unmatched documents, reproduced the known defect exactly, and surfaced one document the prior audit had not named — which is what a reconciliation is supposed to do.

18 Implementation plan

Sequenced by dependency and by cost of delay, not by size. Items 1–2 are hazards; 3–5 are prerequisites for trusting anything the pipeline reports; 6 is the decision that actually determines how big this programme is.

#	ACTION	WHY NOW	EFFORT
1	Disarm <code>work/cleanup_pilot.py</code> — require <code>--apply</code> , scope it to an explicit id list, and fix the false docstring	One careless invocation destroys 9,377 bills. Nothing else in this list can undo that	MINUTES
2	Block <code>goods-post</code> until item 3 lands	1,468 foreign-currency documents are contact-ready and blocked on nothing; the next run understates ₹38.18 crore	MINUTES
3	Fix the currency handling — carry <code>currency</code> and <code>fx_rate</code> into <code>bill.currency / conversionRate</code> with <code>purchaseType=INTERNATIONAL</code> (not a swap to <code>ap_amount_hc</code>)	The target already models this and the backend already honours it. Swapping columns instead would import the N1 corruption and discard the currency fact	SMALL

#	ACTION	WHY NOW	EFFORT
4	Re-extract the staging mirror on the unstripped invoice number , then reconcile the mirror against live <code>AP0BL</code> per document	Until this is done, no amount sourced from <code>ap_amount_hc</code> can be trusted, and the size of the damage is only bounded, not known	SMALL
5	Persist the crosswalk into <code>idedat_staging.crosswalk</code> , which exists and holds 0 rows; keep the file as a cache	Single point of irreversible loss; the destination already exists	SMALL
6	Give the reconciler an independent source — expected values from live Sage, never from the staging column the loader read	Closes the blind spot that let a ₹48 crore error read green	MEDIUM
7	Reproduce or retire the ₹8.72 Cr mis-headed-ledger finding	It is ranked HIGH and drives a proposed correcting journal, and I could not reproduce it. Acting on an unreproduced finding moves real money the wrong way	SMALL
8	Put scope to the business: sales, payments, opening balances, credit/debit notes, prepayments, inventory — in or out?	This determines whether the programme is 60% remaining or 5× its current size. It has never been asked	DECISION
9	Source the CIN/LLPIN file	Highest-leverage non-engineering item; genuinely absent from Sage, so no amount of code will find it	EXTERNAL
10	Then resume <code>goods-post</code> at scale, with per-line GST reconciliation in place	14,628 goods documents have never had a per-line tax check; this is the stream about to run at volume	MEDIUM

What not to do

- › **Do not switch the goods path to** `ap_amount_hc` **as previously recommended** without fixing the extract key first.
- › **Do not re-apply** `sage-migration-backend-changes.patch` — it is missing 12 of the 32 reverted files and reintroduces a silent RCM default.
- › **Do not raise a tolerance to make a check pass.** The –₹1.38 net drift across 2,693 documents is genuine rounding and is not accumulating; the 901 RCM differences are structural and should be reported separately, not tolerated away.
- › **Do not re-post 5,317 vouchers** to correct a ledger classification — if the mis-heading is confirmed, correct it with a journal.
- › **Do not treat the blended 34.2% as progress.** Report the two streams separately; they are in completely different states.

19 Remaining unknowns

Stated plainly, because a migration plan that hides its unknowns is worse than one that has them.

UNKNOWN	WHAT WOULD SETTLE IT
Whether the ₹8.72 Cr mis-heading is real , and if so which head is correct	The prior audit's original query; then a finance judgement on which of Direct or Indirect is right — the data cannot decide this
Whether the four scope exclusions are deliberate — ₹104.43 Cr, including ₹28.78 Cr in "Difference Adjustment Control A/C"	Someone who knows what that account is for. This is a business question wearing a data costume
The correct treatment of 43 foreign-currency AP-direct invoices (₹25.42 Cr) excluded by the INR filter	A scope decision. With item 3 implemented, including them becomes technically straightforward
Whether migrated payables can ever tie to Sage's	They cannot until opening balances and payments load: ₹497.2M migrated against a Sage opening of ₹535.1M and closing of ₹429.5M — the migrated figure is neither, and should not be expected to be
Whether the 1L90 related-party accounts (₹3.26 bn) are payables	A question about how the business thinks, not about the data. Including or excluding them moves the opening-balance gap by an order of magnitude
Whether the 1,079 no-distribution documents are loan repayments (₹295,107,559.76)	The count and value reproduce exactly; the interpretation does not. Trace two through APOBP and GLPOST
How OL order IDs join to OE documents , and the full sales posting chain	Only needed if sales comes into scope — but must be done before, not during
Whether the 289 blocked vendors are genuinely held or merely not yet attempted	Re-run the masters phase against current code and count what actually fails

20 Final recommendation

Treat this as a working pilot that has proven its method on one document type, not as a migration that is two-thirds finished.

The evidence supports an unusually clear split. **The hard part is done and it is right.** Indian GST, reverse charge, rate snapping, receipt-split consolidation, ledger construction, double-entry balance — all of it reproduces against live Sage, and where it differs it differs by rounding or by a documented structural divergence. Three non-RCM documents out of 9,376 differ by a rupee or more, and two of those are one known bug with a known cause. That is a good result and the people who built it should be told so.

The easy parts are what is missing. Not because they are technically hard, but because they were never scoped: sales, purchase orders, receipts, payments, notes, opening balances, inventory. The programme has been reporting a percentage against a denominator that only counts purchase invoices, which makes 34.2% sound like progress toward an end state rather than progress through one of eight objects.

The single most important thing to change

Stop reporting one completeness number. **AP-direct expense bills are 72.4% across; the goods stream is 0.88%; everything else is 0%.** Those three facts lead to three completely different conversations, and averaging them into one produces a number that supports none of them. Every decision in the plan above follows from telling them apart.

And before anything else runs: disarm the delete script, and do not let `goods-post` reach the API until the currency handling is fixed — with the real currency carried, not with the column the prior audit recommended, which is itself corrupted on at least 174 documents.

Method and posture. Every statement issued against both databases was a `SELECT`, `SHOW` or `DESCRIBE`; no migration phase was run, not even `dryrun`, which reaches the live API. No backend code, script, configuration or data was modified. The one write performed was `work/find_sage.py --write` updating `SQL_HOST` in `.env`, because the Sage laptop had moved to a new DHCP address and was otherwise unreachable. Figures are as at 8 September 2026 against Sage `IDEDAT` and SMEAssist org `wonderblues`. Where a prior claim could not be reproduced it is marked **NOT REPRODUCED** rather than refuted; failing to reproduce a finding is not the same as disproving it.